LOVELY PROFESSIONAL UNIVERSITY

Transforming Education Transforming India

Course Code: INT-315	**Course Title: Cluster Computing**
Academic Task No: 02	**Academic Task Title: INSURANCE CLAIM PREDICTION**

Name of the faculty member: Aman Singh Rayat

Submitted by: Atul Sharma

Roll no: 33

Github-https://github.com/AtulSharma001/insurance-claim-prediction

INSURANCE CLAIM PREDICTION USING PYSPARK

Abstract

Insurance systems play a critical role in modern financial and risk management industries. With the increasing volume of policyholders and claims, it becomes essential for insurance companies to adopt intelligent systems that can predict claim behavior accurately and efficiently. Traditional methods of claim analysis are often manual, time-consuming, and unable to scale with large datasets.

This project focuses on the application of machine learning and distributed data processing using Apache Spark to analyze and predict insurance claims. The dataset consists of various attributes related to customers, vehicles, and policies, such as age, vehicle specifications, safety features, and region-based information. By leveraging PySpark, a scalable data processing framework, the project efficiently handles large datasets and performs preprocessing, feature engineering, and model training.

The machine learning models implemented in this project include Logistic Regression and Random Forest Classifier. These models are used to classify whether a customer is likely to make an insurance claim. Special attention is given to handling class imbalance using weighted learning techniques to improve model performance on minority classes.

In addition to predictive modeling, the project includes exploratory data analysis and visualization using Matplotlib. Various factors such as fuel type, transmission type, and customer age are analyzed to understand their relationship with claim occurrence.

The results demonstrate that machine learning models, when combined with proper data preprocessing and feature engineering, can effectively predict insurance claims with high accuracy. The study highlights the importance of data-driven decision-making in the insurance sector and shows how scalable technologies like Apache Spark can be used to build efficient and robust predictive systems.

I. Introduction

I.1 Background of Insurance Systems

Insurance systems are essential components of modern economies, providing financial protection against risks and uncertainties. With the rapid increase in vehicle ownership and policyholders, insurance companies are required to process a massive number of claims efficiently. Traditional claim processing systems often rely on manual evaluation and predefined rules, which may lead to inefficiencies, delays, and inaccuracies.

In recent years, the integration of data-driven approaches has transformed the insurance industry. Organizations now collect large volumes of data related to customers, vehicles, and claims, which can be utilized to improve decision-making processes. However, extracting meaningful insights from such data requires advanced computational techniques.

I.2 Need for Predictive Analytics

Predictive analytics plays a crucial role in identifying patterns and trends within large datasets. In the context of insurance, predicting whether a customer will file a claim helps companies manage risk, reduce losses, and design better policies.

Without predictive models, insurance companies face challenges such as fraud detection, inefficient premium pricing, and delayed claim processing. Machine learning provides a systematic approach to analyze historical data and make accurate predictions, thereby improving operational efficiency.

I.3 Machine Learning in Insurance

Machine learning has emerged as a powerful tool in the insurance sector. It enables automated decision-making by learning patterns from data rather than relying solely on human intervention. Classification algorithms, such as Logistic Regression and Random Forest, are widely used to predict claim outcomes.

These models analyze various features such as customer demographics, vehicle characteristics, and safety measures to determine the likelihood of a claim. By using machine learning, insurance companies can optimize their processes and enhance customer satisfaction.

I.4 Why Use PySpark

PySpark is a distributed data processing framework that allows efficient handling of large-scale datasets. Unlike traditional tools, PySpark can process data in parallel, significantly reducing computation time.

It provides built-in machine learning libraries that simplify tasks such as feature transformation, model training, and evaluation. The scalability and speed of PySpark make it an ideal choice for handling insurance datasets, which often contain thousands or millions of records.

I.5 Purpose of the Study

The primary purpose of this study is to develop a machine learning model capable of predicting insurance claims using PySpark. The project also aims to analyze the relationship between different features and claim outcomes through visualization and statistical analysis.

I.6 Scope of the Study

The scope of this project is limited to analyzing structured insurance data and building a predictive model for claim classification. The study focuses on supervised learning techniques where historical data is used to train models for predicting future outcomes. The implementation is carried out using PySpark, which allows efficient handling of medium to large-scale datasets.

The project does not include real-time streaming data or unstructured data such as images or text. However, the methodology can be extended to incorporate

such data sources in the future. The scope also includes exploratory data analysis to understand feature relationships and their impact on claim occurrence.

II. Objectives

The primary objective of this project is to develop a robust machine learning model capable of predicting whether an insurance claim will occur. This involves understanding the dataset, preprocessing the data, and selecting appropriate algorithms.

Another objective is to analyze how different features contribute to claim prediction. This includes identifying which variables have a strong influence and which have minimal impact. Understanding these relationships is important for improving model accuracy and interpretability.

The project also aims to evaluate model performance using multiple metrics. Instead of relying solely on accuracy, other measures such as precision, recall, and F1 score are considered to ensure a balanced evaluation, especially in the presence of imbalanced data.

III. Dataset Description

The dataset used in this project contains detailed information about insurance policies, vehicles, and customers. It includes both numerical and categorical features, making it suitable for machine learning analysis.

Key attributes include customer age, vehicle age, fuel type, transmission type, engine specifications, safety features such as airbags and electronic stability control, and regional information. The target variable is claim_status, which indicates whether a claim was made.

The dataset is imbalanced, with a significantly higher number of non-claim records compared to claim records. This imbalance poses a challenge for model training and requires special handling techniques.

IV. Methodology

I Data Preprocessing

Data preprocessing is a critical step in the machine learning pipeline. In this project, missing values were handled by either removing incomplete rows or filling them with appropriate values such as averages.

Unnecessary columns that do not contribute to prediction were removed to improve model efficiency. Additionally, binary categorical features were converted into numerical format to make them suitable for machine learning algorithms.

II Feature Engineering

Feature engineering involves transforming raw data into meaningful features that improve model performance. Categorical variables such as fuel type and transmission type were converted into numerical representations using String Indexing and One-Hot Encoding.

All features were then combined into a single vector using VectorAssembler, which serves as input to the machine learning model.

III Handling Class Imbalance

The dataset exhibited a significant imbalance between claim and non-claim cases. To address this issue, a weight column was introduced, assigning higher importance to minority class instances.

This approach ensures that the model pays more attention to claim cases during training, improving its ability to detect them.

IV Model Building

Two machine learning models were implemented in this project: Logistic Regression and Random Forest Classifier. Logistic Regression provides a simple and interpretable baseline, while Random Forest offers improved performance by capturing non-linear relationships.

A pipeline was created to integrate all preprocessing steps and the model into a single workflow, ensuring consistency and efficiency.

V Evaluation Metrics

The model was evaluated using multiple metrics, including accuracy, precision, recall, and F1 score. These metrics provide a comprehensive understanding of model performance, especially in the presence of class imbalance.

VI. Results and Analysis

The model achieved high accuracy, indicating that it performs well in predicting non-claim cases. However, due to class imbalance, the model tends to predict fewer claim cases.

Analysis of visualizations revealed that factors such as fuel type and transmission type do not significantly affect claim rates individually. Instead, claims depend on a combination of multiple features.

Customer age and safety features showed some variation in claim distribution, suggesting that these factors may have a stronger influence on claim behaviour.

VII. Advantages of the Approach

This approach provides a scalable solution for insurance claim prediction. The use of PySpark ensures efficient processing of large datasets, while machine learning models enable accurate predictions.

The integration of visualization techniques enhances the interpretability of results, allowing better understanding of data patterns. Additionally, the use of pipelines simplifies the implementation and ensures consistency.

VIII. Limitations

One of the main limitations of this project is the imbalance in the dataset, which affects the model's ability to predict claim cases accurately. Another limitation is the use of a limited dataset, which may not capture all real-world scenarios.

The model also relies on structured data and does not consider external factors such as driving behaviour or environmental conditions.

IX. Future Enhancements

Future improvements can include the use of advanced techniques such as oversampling or SMOTE to handle class imbalance more effectively. Additional features can be incorporated to improve prediction accuracy.

More advanced models such as Gradient Boosting or Neural Networks can be explored. Integration with real-time data systems can also enhance the applicability of the model in real-world scenarios.

X. Conclusion

This project demonstrates the effectiveness of machine learning and distributed computing in predicting insurance claims. By leveraging PySpark, the system efficiently processes large datasets and builds scalable models.

The results highlight the importance of feature engineering and proper handling of class imbalance in achieving accurate predictions. Although the model performs well overall, further improvements can be made to enhance its ability to detect claim cases.

The study emphasizes the growing importance of data-driven approaches in the insurance industry and provides a foundation for developing more advanced predictive systems in the future.

Appendix: Model Workflow Explanation

The workflow of the project begins with data loading and preprocessing, followed by feature transformation and model training. The trained model is then evaluated using multiple metrics, and the results are visualized for better interpretation.

Each stage in the workflow is interconnected, ensuring a smooth transition from raw data to final predictions. This structured approach improves reliability and makes the system easier to maintain and extend.

Refrences

1. Apache Spark Documentation,
   Apache Spark Official Documentation. Available at:
   https://spark.apache.org/docs/latest/

2. PySpark Documentation,
   PySpark API Reference. Available at:
   https://spark.apache.org/docs/latest/api/python/

3. Machine Learning,
   Tom M. Mitchell,
   Machine Learning, McGraw-Hill, 1997.

4. Pattern Recognition and Machine Learning,
   Christopher M. Bishop, Springer, 2006.

5. Logistic Regression,
   David W. Hosmer & Stanley Lemeshow,
   Applied Logistic Regression, Wiley, 2000.

6. Random Forest,
   Leo Breiman,
   “Random Forests,” Machine Learning Journal, 2001.

7. Matplotlib Documentation,
   Available at: https://matplotlib.org/

8. Data Mining Concepts and Techniques,
   Jiawei Han, Micheline Kamber, Morgan Kaufmann, 2011.

9. Class Imbalance,
   Haibo He & Edwardo A. Garcia,
   “Learning from Imbalanced Data,” IEEE Transactions, 2009.

Source Code:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Insurance Claim Prediction") \
    .getOrCreate()
df = spark.read.csv(
    "C:/Users/satul/Downloads/Insurance claims data.csv",
    header=True,
    inferSchema=True
)

df = df.limit(500)
df.show(5)
from pyspark.sql.functions import col

df = df.dropna()
```

```
df = df.fillna({
    "subscription_length": 0,
    "vehicle_age": 0,
    "customer_age": df.selectExpr("avg(customer_age)").first()[0]
})
from pyspark.sql.functions import when

binary_cols = [
    "is_esc", "is_adjustable_steering", "is_tpms",
    "is_parking_sensors", "is_parking_camera",
    "is_front_fog_lights", "is_rear_window_wiper",
    "is_rear_window_washer", "is_rear_window_defogger",
    "is_brake_assist", "is_power_door_locks",
    "is_central_locking", "is_power_steering",
    "is_driver_seat_height_adjustable",
    "is_day_night_rear_view_mirror",
    "is_ecw", "is_speed_alert"
]

for c in binary_cols:
    df = df.withColumn(c, when(col(c) == "Yes", 1).otherwise(0))
df = df.drop("policy_id", "max_torque", "max_power", "engine_type")
df = df.withColumn(
    "weight",
    when(col("claim_status") == 1, 2).otherwise(1)
)
from pyspark.ml.feature import StringIndexer, OneHotEncoder

categorical_cols = ["fuel_type", "transmission_type", "segment", "model"]

indexers = [
    StringIndexer(inputCol=c, outputCol=c+"_index")
```

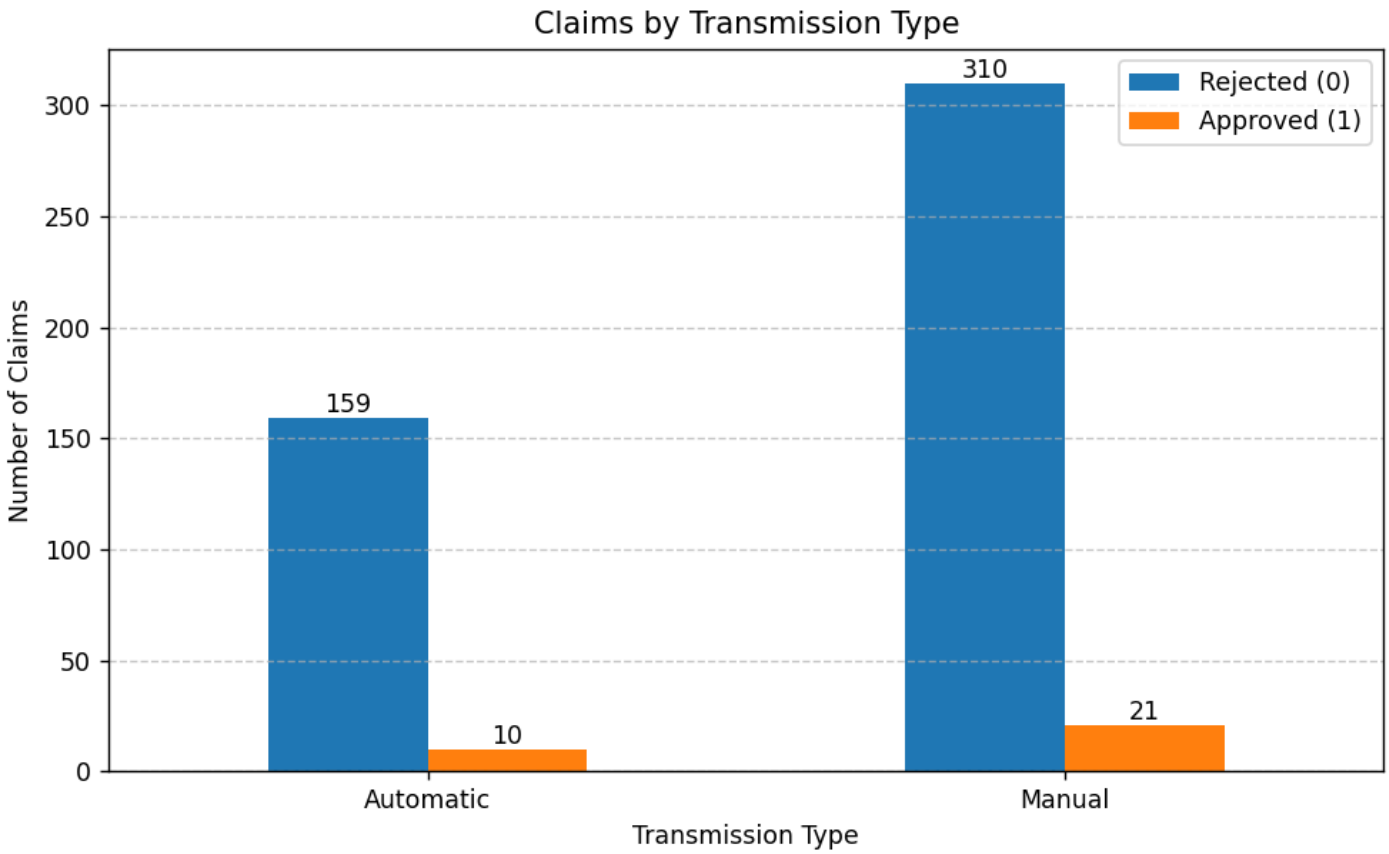
```
    for c in categorical_cols
]

encoders = [
    OneHotEncoder(inputCol=c+"_index", outputCol=c+"_vec")
    for c in categorical_cols
]
from pyspark.ml.feature import VectorAssembler

numerical_cols = [
    "subscription_length", "vehicle_age", "customer_age",
    "region_density", "displacement", "cylinder",
    "length", "width", "gross_weight", "ncap_rating"
] + binary_cols

feature_cols = numerical_cols + [c+"_vec" for c in categorical_cols]

assembler = VectorAssembler(
    inputCols=feature_cols,
    outputCol="features"
)
from pyspark.ml.classification import LogisticRegression

lr = LogisticRegression(
    featuresCol="features",
    labelCol="claim_status",
    weightCol="weight"
)
from pyspark.ml import Pipeline

pipeline = Pipeline(stages=indexers + encoders + [assembler, lr])
train_data, test_data = df.randomSplit([0.8, 0.2], seed=42)
```

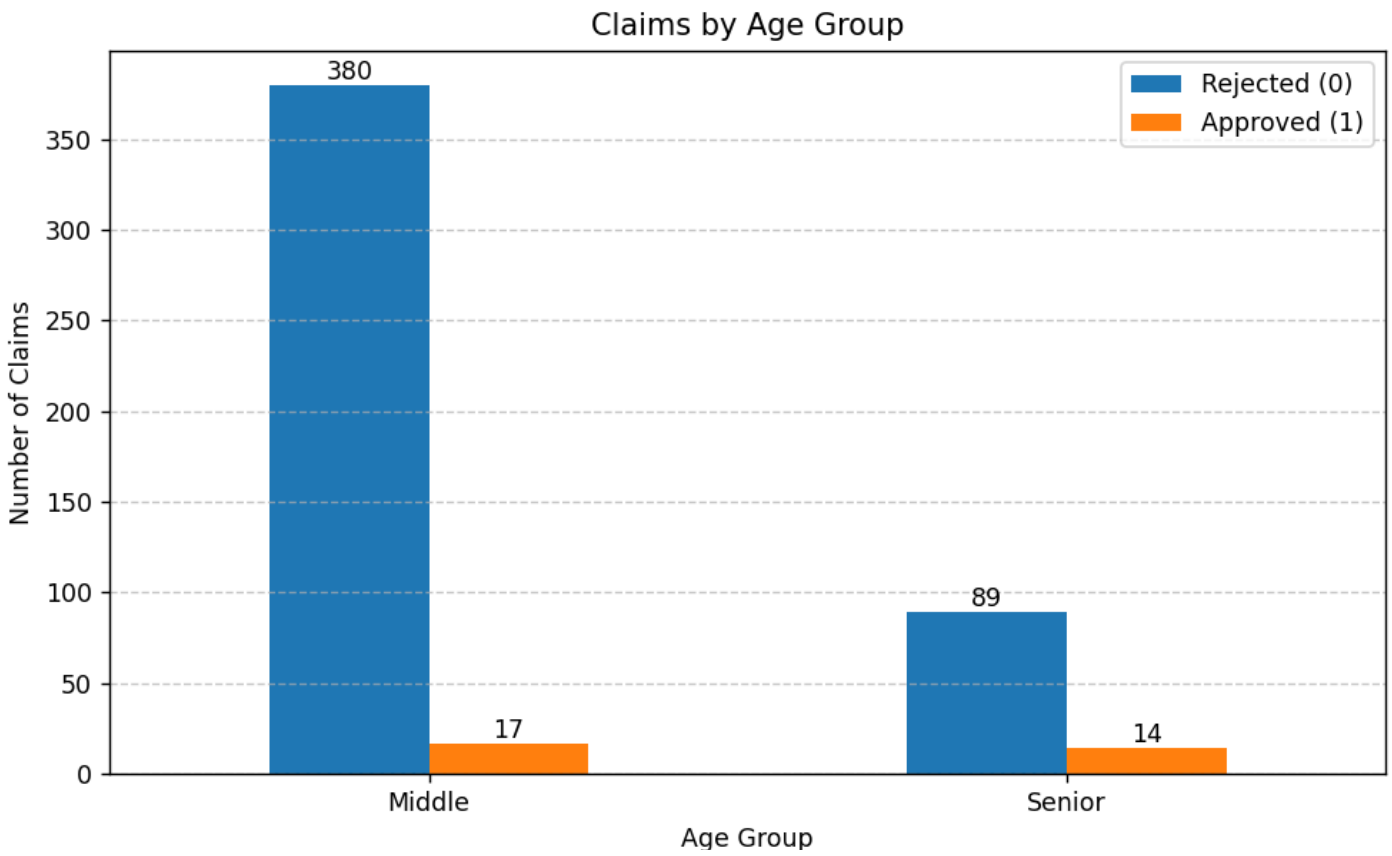
```
model = pipeline.fit(train_data)
predictions = model.transform(test_data)

predictions.select("claim_status", "prediction").show(10)
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

accuracy = MulticlassClassificationEvaluator(
    labelCol="claim_status",
    predictionCol="prediction",
    metricName="accuracy"
).evaluate(predictions)

f1 = MulticlassClassificationEvaluator(
    labelCol="claim_status",
    predictionCol="prediction",
    metricName="f1"
).evaluate(predictions)

precision = MulticlassClassificationEvaluator(
    labelCol="claim_status",
    predictionCol="prediction",
    metricName="weightedPrecision"
).evaluate(predictions)

print("Accuracy:", accuracy)
print("F1 Score:", f1)
print("Precision:", precision)
predictions.groupBy("claim_status", "prediction").count().show()
import matplotlib.pyplot as plt

pdf = df.groupBy("fuel_type", "claim_status").count().toPandas()
```

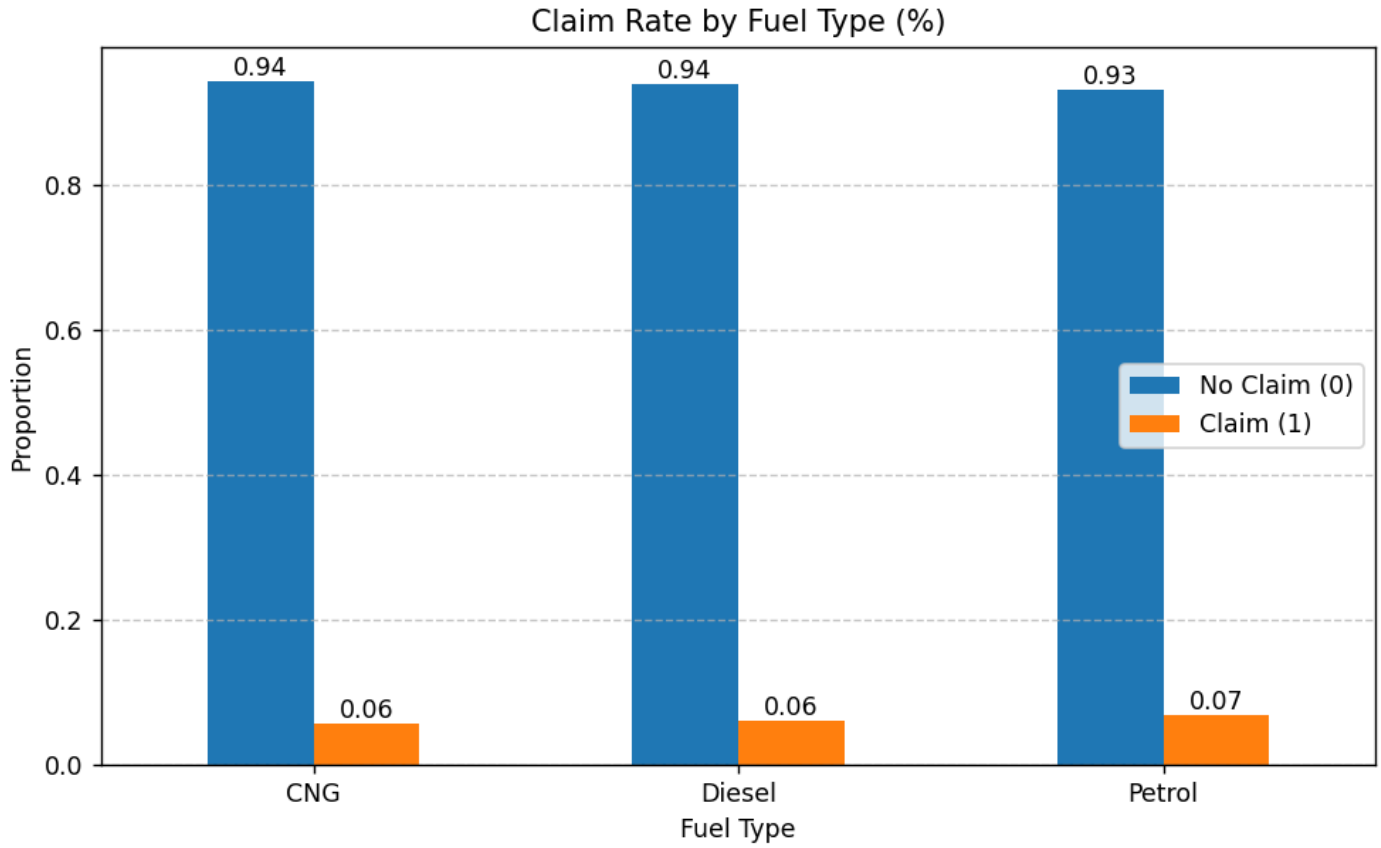
```
pivot_df = pdf.pivot(index="fuel_type", columns="claim_status", values="count").fillna(0)

ax = pivot_df.plot(kind="bar", figsize=(9,5))

for container in ax.containers:
    ax.bar_label(container, padding=3)

plt.title("Insurance Claim Distribution by Fuel Type")
plt.xlabel("Fuel Type")
plt.ylabel("Number of Claims")
plt.xticks(rotation=0)
plt.legend(["Rejected (0)", "Approved (1)"])
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
df = df.withColumn(
    "age_group",
    when(col("customer_age") < 30, "Young")
    .when(col("customer_age") < 50, "Middle")
    .otherwise("Senior")
)

pdf = df.groupBy("age_group", "claim_status").count().toPandas()

pivot_df = pdf.pivot(index="age_group", columns="claim_status", values="count").fillna(0)

ax = pivot_df.plot(kind="bar", figsize=(8,5))

for container in ax.containers:
    ax.bar_label(container)

plt.title("Claims by Customer Age Group")
```

```
plt.xlabel("Age Group")
plt.ylabel("Number of Claims")
plt.xticks(rotation=0)
plt.legend(["Rejected (0)", "Approved (1)"])
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
pdf = df.groupBy("airbags", "claim_status").count().toPandas()

pivot_df = pdf.pivot(index="airbags", columns="claim_status", values="count").fillna(0)

ax = pivot_df.plot(kind="bar", figsize=(8,5))

for container in ax.containers:
    ax.bar_label(container)

plt.title("Claims by Number of Airbags")
plt.xlabel("Airbags")
plt.ylabel("Number of Claims")
plt.legend(["Rejected (0)", "Approved (1)"])
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

Ouput Screenshots:




Claims by Age Group
Rejected (0)
Approved (1)
Number of Claims
0
50
100
150
200
250
300
350
380
17
89
14
Middle
Senior
Age Group


Claim Rate by Fuel Type (%)
No Claim (0)
Claim (1)
Proportion
0.0
0.2
0.4
0.6
0.8
0.94
0.06
0.94
0.06
0.93
0.07
CNG
Diesel
Petrol
Fuel Type